

UFO_s: An Ultra-fast Toolkit for Multiparty Computation on Small Elements

Jiacheng Gao[✉], Moyang Xie[✉], Yuan Zhang[✉], *Member, IEEE*, Sheng Zhong[✉], *Fellow, IEEE*

Abstract—In many secure multiparty computation (MPC) applications, the data to be processed are much smaller than the underlying field or ring size. The encoding domain is typically chosen to be large enough to guarantee security (e.g., a 128-bit prime field for 128-bit security), whereas the actual data may consist of only a few bits, such as 4-bit values in a 16-category classification task. This mismatch can lead to substantial communication and computation overhead in existing MPC protocols, which typically treat data of different ranges uniformly.

We introduce UFO_s, an ultra-fast toolkit for MPC on small elements. UFO_s provides highly optimized protocols for three fundamental primitives: one-hot encoding, comparison, and digit decomposition. While these primitives are designed for small elements, they can also be used to construct higher-level protocols; as an example, we develop a radix sort protocol for sorting large field elements. Experimental results show significant performance improvements over state-of-the-art MPC implementations. In particular, our sorting protocol achieves up to a $58\times$ speedup in the online phase when sorting 2^{16} elements among five parties.

Index Terms—Multi-party computation, one-hot encoding, secure comparison, digit decomposition, secure sorting.

I. INTRODUCTION

Secure multiparty computation (MPC) enables multiple parties to jointly compute a function over their private inputs while keeping those inputs secret. This powerful cryptographic technique has found applications across various domains, including privacy-preserving data analysis [1], secure voting systems [2], confidential financial computations [3], and federated learning [4].

In a typical MPC setting, the data to be processed are often encoded as elements in a finite field $\mathbb{F}_p$ or ring $\mathbb{Z}_p$, where p is a large modulus, (e.g., a 128-bit prime for 128-bit security). For example, the MP-SPDZ framework [5] uses a 128-bit prime field as its default configuration, while ABY³ [6] performs computations over the ring $\mathbb{Z}_{2^{64}}$. Such choices ensure security against attacks and avoid arithmetic overflows [7, 8, 9].

However, in many practical applications, the actual data domain is far smaller than the underlying encoding space, and this mismatch is pronounced in real-world deployments. For instance, Wang et al. [4] extend Crypten [10] to construct a secure machine learning framework for classification tasks. Their method uses an MPC comparison protocol to encode features into one-hot vectors. However, a mismatch arises: the dataset has at most 15 categories (4 bits), while Crypten operates on a 64-bit integer ring $\mathbb{Z}_{2^{64}}$. This leads to at least 60 wasted

bits per comparison, as only 4 bits determine the result, yet the protocol processes full 64-bit elements. Consequently, this size gap causes significant communication and computation overhead in MPC applications, an unnecessary cost overlooked by most prior work.

In this work, we focus on efficient MPC protocols for small elements. These are elements whose sizes are much smaller than the underlying field or ring. We present UFO_s, an ultra-fast toolkit for multiparty computation on small elements. UFO_s contains highly optimized protocols for three fundamental tasks: one-hot encoding, comparison, and digit decomposition. One-hot encoding and comparison are widely used in classification and transformer-based machine learning tasks [4, 10, 11, 12]. While the first two primitives are designed specifically for small elements, we also provide a digit decomposition protocol that decomposes large field elements into their base- t representation, for small t .

The efficiency of UFO_s stems from several key insights. Prior one-hot encoding constructions rely on random one-hot vectors, whose generation forms the main bottleneck: for example, [13] first generates random bits and then converts them into a one-hot vector. To address this, we propose a more efficient approach that samples a random integer and directly demultiplexes it into a one-hot vector, requiring only $2t$ multiplications and achieving improved round complexity. In addition, we propose a fundamentally new approach to one-hot encoding that avoids random one-hot vectors altogether and instead constructs the encoding using a technique based on Lagrange interpolation. Building on efficient one-hot encodings, we observe that several protocols based on binary representations [14, 15, 16, 17] can instead be implemented using one-hot representations, thereby dispensing with many bit operations. As an example, we design an efficient integer secure comparison protocol using one-hot representations. Some prior digit decomposition protocols, such as [17], lack an efficient integer comparison protocol and therefore rely on bit expansion followed by bit-wise comparisons, which leads to a large number of bit operations. By leveraging our integer comparison protocol, this bottleneck can be eliminated, enabling a more efficient digit decomposition.

To demonstrate the power of our techniques, we construct a radix sort protocol for large field elements. By operating on base- t digits instead of individual bits, our approach significantly reduces the number of shuffle invocations compared with prior bit-level designs [18, 19, 20, 21]. Our experiments show significant performance improvements over state-of-the-art constructions. In particular, our sorting protocol achieves up to a $58\times$ speedup in the online phase when sorting 2^{16} elements among five parties.

Jiacheng Gao, Moyang Xie, Yuan Zhang, and Sheng Zhong are with the State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing 210023, China (e-mail: jcgao@smail.nju.edu.cn, xie_moyang@foxmail.com, zhangyuan@nju.edu.cn, zhongsheng@nju.edu.cn).

Jiacheng Gao and Moyang Xie contributed equally to this work.

Our contributions are summarized as follows:

- 1) We present highly optimized MPC protocols for small elements, including one-hot encoding, comparison, and digit decomposition. These protocols achieve significant performance improvements over existing constructions.
- 2) We design a radix sort protocol based on our small-element primitives, which efficiently sorts large field elements while significantly reducing communication and computation costs compared with existing MPC radix sort protocols.
- 3) We analyze the complexity of our protocols, implement them, and evaluate their performance through extensive experiments. The results demonstrate the efficiency and practicality of our approach for real-world MPC applications.

The rest of this paper is organized as follows. Section II reviews related work. Section III introduces the preliminaries. Section IV presents our one-hot encoding protocol. Section V presents our digit decomposition protocol. Section VI describes our radix sort protocol. Section VII reports the experimental evaluation. Finally, Section VIII concludes the paper.

II. RELATED WORK

A. Related Works on One-hot Encoding

Despite the widespread use of one-hot vectors, only a few works study one-hot encoding in multiparty computation. Attrapadung et al. [22] proposed a two-party protocol based on oblivious transfer (OT). Morita et al. [13] proposed a protocol based on additive and replicated secret sharing. Due to the use of replicated secret sharing, their protocol supports only a small number of parties unless the number of corrupted parties is bounded by a small constant (e.g., ≤ 2).

These works share a common approach. To encode a secret-shared integer $0 \leq x < t$ into a one-hot vector of length t , the parties first generate a secret-shared random one-hot vector $\mathbf{b} = (b_0, \dots, b_{t-1})$ with exactly one $b_i = 1$ in the offline phase. In the online phase, they open $d = x - i \bmod t$ and locally shift $\mathbf{b}$ by d positions so that the 1 moves to position x . To generate such a random one-hot vector, Attrapadung et al. [22] rely on OT, where the sender prepares t random one-hot vectors of length t and the receiver obliviously retrieves one, incurring $O(t^2)$ computation. Morita et al. [13] generate a random one-hot vector using $\log t$ random bits, $2t$ multiplications, and $O(\log t)$ rounds. Although the randomness usage is optimal, the round complexity is high. Furthermore, [22] assume that x and i are shared in $\mathbb{Z}_t$, while [13] require t to be a power of two. These assumptions limit applicability, since in many settings x is shared in a large field $\mathbb{Z}_p$ and t may not be a power of two.

To address these limitations, we show how to generate a random one-hot vector with $2t$ multiplications and $O(\log \log t)$ rounds, scalable to any number of parties. We also present a direct construction that produces a one-hot vector for a given index with $2t$ multiplications and $O(1)$ rounds, without random bits or preprocessing. Neither construction assumes any special form of t , nor requires x to be shared in $\mathbb{Z}_t$.

B. Related Works on Decomposition

The most widely used decomposition method is **bit decomposition**, which decomposes a secret-shared integer into its secret-shared binary representation. It was introduced by Damgård et al. [23], who also applied it to integer comparison and exponentiation. The key idea is to mask the input integer with random bits, open the masked value and decompose it in plaintext. The decomposed bits are unmasked via bitwise addition with the random bits. It was subsequently improved by Toft et al. [24], Reistad et al. [9], and Veugen et al. [25]. However, bit decomposition remains expensive, and many MPC protocols avoid it, e.g., the comparison protocols in [8, 14, 15, 16]. Nevertheless, bit decomposition remains useful for certain non-linear operations [23] and in some cases speed up computations (e.g., in radix sort [20]).

In this paper we consider **digit decomposition**, a generalization of bit decomposition, which decomposes an integer into its representation in base t for any integer $t > 1$. Existing works mainly study digit decomposition in the two-party additive secret sharing setting [26, 27, 28], typically assuming that t is a power of 2. In this case, parties can locally truncate their shares into l -bit chunks (for $t = 2^l$), and then compute the carry bits to correct the chunks.

The closest related work is by Ning et al. [17], which also studies digit decomposition in the multiparty setting. Given an L -bit integer, their protocol decomposes it in base t with digit length $l = \lceil \log t \rceil$. The approach follows the masking paradigm used in bit decomposition: the parties generate random digits, mask the input, open the masked value, decompose it in plaintext, and then remove the mask. Instead of performing digit-wise subtraction directly, their protocol converts the result into a *digit-binary representation*, where each digit is represented in l binary bits, yielding $\lceil L/l \rceil$ digits in total. The mask is then removed via bitwise subtraction on the resulting binary strings of length $\lceil L/l \rceil \cdot l$. This conversion avoids digit-wise subtraction, which is difficult to implement securely because it requires handling carries between digits, typically involving secure comparison with the constant t .

In our construction, we directly perform digit-wise subtraction to remove the mask. This is enabled by our one-hot encoding protocol, which provides a efficient comparison functionality. In this way, our protocol only requires bit-wise subtraction of two $(\lceil L/l \rceil)$ -bit strings, avoiding the expansion of each digit into l binary bits. Consequently, the number of required bit subtractions is reduced to only $1/l$ of that in [17].

C. Related Works on MPC Sorting

Sorting, as a key primitive in many protocols, has been extensively studied in secure computation. To implement a sorting protocol in multiparty computation, currently there are three main approaches, which we now discuss.

A straightforward approach is to directly implement a **comparison-based sorting** algorithm. Comparison-based sorting protocols are typically built from atomic compare-swap operations, which compare two elements and conditionally swap them. This approach is conceptually simple but generally inefficient. Since comparison outcomes cannot be re-

vealed, efficient data-dependent algorithms such as quicksort and mergesort are not applicable. Existing constructions either require $O(n^2)$ comparisons (e.g., [29, 30]), or require $O(n \log^2 n)$ comparisons (e.g., [31, 32]), or require $O(n \log n)$ comparisons but only sort correctly with some probability (e.g., [33, 34, 35]), or require $O(n \log n)$ comparisons but with prohibitively large constants (e.g., [36, 37]).

To enable more efficient data-dependent sorting, Hamada et al. [38] proposed the **shuffle-then-sort** paradigm, which shuffles the input before sorting. This allows comparison outcomes to be safely revealed, as the correspondence between positions and elements is concealed.

Given the high cost of secure comparison, **radix sort** has been widely studied in MPC to avoid comparison operations. Based on the “shuffle-then-reveal” technique, Bogdanov et al. [39] and Laud et al. [19] proposed radix sort protocols that takes a bitwise-shared array as input. They process the keys from the least significant bit (LSB) to the most significant bit (MSB), shuffling and revealing one bit per round. The communication cost is $O(nl^2)$, where l is the bit length of the keys. Hamada et al. [20] reduced this to $O(nl)$ by shuffling only constant-length vectors per round, and Chida et al. [21] further optimized the design.

Although radix sort improves efficiency in MPC, existing constructions [18, 19, 20, 21] suffer from round complexity linear in the bit-length l , as they require bit-decomposed keys and LSB-to-MSB processing. If each shuffle invocation costs c rounds, the total becomes lc . Moreover, c may not be constant; for example, Keller and Scholl’s shuffle [40] has $c = O(m \log n)$ for m parties and n items, yielding an overall round complexity of $O(ml \log n)$. Hence, current MPC radix sort protocols scale poorly with l .

To address this limitation, we apply our digit decomposition protocol to represent the input keys in base t (e.g., $t = 16$ or 256), and use one-hot encoding to perform radix sort directly on the base- t digits. This reduces the number of rounds and shuffles by a factor of $\log t$. Although one-hot encoding introduces additional overhead, the reduced shuffling significantly lowers the communication and computation costs, resulting in improved overall performance.

III. PRELIMINARIES

A. Basic Notations

Let m parties $P_1, \dots, P_m$ participate in the computation, and suppose there are n integers to be processed (e.g., shuffled or sorted depending on the context), each of bit length L . Let κ denote the security parameter, and let $\mathbb{Z}_p$ be a finite field of prime order p . Throughout the paper, secret values are encoded in $\mathbb{Z}_p$ with $p > 2^L$.

We assume p is sufficiently large to avoid modular wrap-around during the computation; specifically, $\log p > L + \lceil \log m \rceil + \kappa$. This assumption is standard in MPC (e.g., [8, 9]).

For integers $a \leq b$, let $[a, b] := \{a, a+1, \dots, b\}$ and $[a] := \{0, 1, \dots, a-1\}$. For a vector $\vec{v}$, denote its i -th entry by v_i . Vector, array, and matrix indices start from zero.

B. Network and Security Model

We assume that every pair of parties is connected by a secure channel providing confidentiality and authentication. No broadcast channel is assumed; thus broadcasting a message requires sending $m-1$ copies.

We consider a semi-honest adversary model with an honest majority (i.e., more than half of the participants are honest), where parties follow the protocol, but may try to learn additional information from the execution and may collude to share data. Our protocols can be extended to the malicious setting if the underlying primitives are maliciously secure. As our contributions are mainly algorithmic, we focus on the semi-honest setting in this paper.

C. Secret Sharing Scheme

Let $\llbracket x \rrbracket$ denote the secret sharing of a value x among all parties. For convenience, we sometimes write $\llbracket x_0, \dots, x_{n-1} \rrbracket$ to denote the secret-shared vector $\llbracket \vec{x} \rrbracket = (\llbracket x_0 \rrbracket, \dots, \llbracket x_{n-1} \rrbracket)$. Our constructions can be instantiated with any linear secret sharing scheme that supports the following operations:

- 1) $\llbracket x \rrbracket \leftarrow \text{Share}(P_i : x)$: party P_i shares a secret x among all parties.
- 2) $x \leftarrow \text{Open}(\llbracket x \rrbracket)$: reconstruct the secret value x to all parties.
- 3) $\llbracket z \rrbracket \leftarrow \text{Add}(\llbracket x \rrbracket, \llbracket y \rrbracket)$: compute $\llbracket z \rrbracket = \llbracket x + y \rrbracket$, where $\llbracket x \rrbracket$ and $\llbracket y \rrbracket$ are secret-shared values. Since the scheme is linear, addition is performed locally.
- 4) $\llbracket z \rrbracket \leftarrow \text{Mul}(\llbracket x \rrbracket, \llbracket y \rrbracket)$: compute $\llbracket z \rrbracket = \llbracket xy \rrbracket$.
- 5) $\llbracket r \rrbracket \leftarrow \text{Rand}()$: sample $r \xleftarrow{\$} \mathbb{Z}_p$ and share it to all parties.
- 6) $\llbracket b \rrbracket \leftarrow \text{RandBit}()$: sample a secret random random bit b and share it (see [8] for an implementation).

For simplicity, we also write $\llbracket x \rrbracket + \llbracket y \rrbracket$ and $\llbracket x \rrbracket \cdot \llbracket y \rrbracket$ to denote $\text{Add}(\llbracket x \rrbracket, \llbracket y \rrbracket)$ and $\text{Mul}(\llbracket x \rrbracket, \llbracket y \rrbracket)$, respectively. Besides these basic operations, we require two additional primitives:

RandSum: $\llbracket r \rrbracket \leftarrow \text{RandSum}()$. We require a primitive that generates a secret-shared random value equal to the sum of m random values, each sampled from $[2^\kappa - 1]$. This randomness is used in our one-hot encoding protocol to hide the encoded index. The same technique also appears in [8, 9]. Concretely, each party P_i samples $r_i \xleftarrow{\$} [2^\kappa - 1]$, shares $\llbracket r_i \rrbracket$ with all parties, and the parties locally compute $\llbracket r \rrbracket = \sum_{i=1}^m \llbracket r_i \rrbracket$.

Shuffle: $\llbracket \mathbf{v}_i \rrbracket_{i \in [n]} \leftarrow \text{Shuffle}(\llbracket \mathbf{u}_i \rrbracket_{i \in [n]})$. An MPC shuffle protocol takes as input secret-shared vectors $\llbracket \mathbf{u}_i \rrbracket_{i \in [n]}$ and outputs a randomly permuted version $\llbracket \mathbf{u}_{\pi(i)} \rrbracket_{i \in [n]}$, where π is a secret permutation over $[n]$. Since π is randomly chosen and unknown to all parties, we do not explicitly denote it. Efficient shuffle protocols are well established in MPC [40, 41, 42]. In this paper we instantiate the protocol of [40], which requires $O(m \log n)$ rounds and $O(m^2 n \log n)$ communication.

IV. ONE-HOT ENCODING

A one-hot vector is a binary vector with exactly one entry being 1. In this section, we give our construction for encoding an integer into a one-hot vector, which serves as a key building block for our subsequent constructions.

Given a secret shared value $\llbracket i \rrbracket$ with $i \in [t-1]$, the one-hot encoding protocol outputs its one-hot representation

$$\llbracket \bar{s} \rrbracket = \llbracket s_0, s_1, \dots, s_{t-1} \rrbracket,$$

with $s_i = 1$ and $s_j = 0$ for all $j \neq i$. We call i the “index” of the one-hot vector, where $s_i = 1$ is the only “hot” bit.

Denote by $\bar{t} := 2^{\lceil \log t \rceil}$. The generation of a one-hot vector can be divided into two phases, an offline one and an online one, which we present in section IV-A and section IV-B, respectively. In the offline phase, the parties generate a random one-hot vector for a uniformly random index in $[\bar{t}-1]$. In the online phase, the parties use any such random one-hot vector, along with its secret shared index, to create the index’s one-hot vector, with slight overhead. This paradigm is similar to the Beaver’s triplets: Most of the computations are done before the input $\llbracket i \rrbracket$ becomes available.

A. Random One-hot Vector

The functionality of this protocol is to generate a one-hot vector with secret index $\llbracket r \rrbracket$ uniformly random in $[\bar{t}-1]$. The output consists of the vector together with its secret-shared index, i.e. tuple $(\llbracket \bar{s} \rrbracket, \llbracket r \rrbracket)$ with $s_r = 1$. Note that both the vector and its index are kept secret in the process.

Note that, if the parties acquire $\log \bar{t}$ many secret shared random bits, the demux protocol due to [43] can output the desired one-hot vector, with index $\llbracket r \rrbracket := \sum_{i=0}^{\log \bar{t}-1} 2^i \llbracket r_i \rrbracket$ uniformly random. See algorithm 1 for its description.

For simplicity of exposition, we assume $t = 2^k$. The Demux protocol has recursion depth $\log \log t$, yielding $\log \log t$ communication rounds. The i -th recursion level contributes roughly $2^{i-1+\log t \cdot 2^{1-i}}$ multiplications as merging overhead. The total number of multiplications is below $2t$, and both computation and communication complexities are $O(tm)$.

Algorithm 1 $(\llbracket s_0 \rrbracket, \dots, \llbracket s_{2^k-1} \rrbracket) \leftarrow \text{Demux}(\llbracket r_0 \rrbracket, \dots, \llbracket r_{k-1} \rrbracket)$

Ensure: The output is a one-hot vector, with index $\sum_{i=0}^{k-1} 2^i \llbracket r_i \rrbracket$.

- 1: **if** $k = 1$ **then**
- 2: Return $(1 - \llbracket r_0 \rrbracket, \llbracket r_0 \rrbracket)$
- 3: **end if**
- 4: $\bar{k} \leftarrow \lfloor k/2 \rfloor$
- 5: $\llbracket b_0 \rrbracket, \llbracket b_1 \rrbracket, \dots, \llbracket b_{2^{\bar{k}}-1} \rrbracket \leftarrow \text{Demux}(\llbracket r_0 \rrbracket, \dots, \llbracket r_{\bar{k}-1} \rrbracket)$
- 6: $\llbracket c_0 \rrbracket, \llbracket c_1 \rrbracket, \dots, \llbracket c_{2^{k-\bar{k}}-1} \rrbracket \leftarrow \text{Demux}(\llbracket r_{\bar{k}} \rrbracket, \dots, \llbracket r_{k-1} \rrbracket)$
- 7: **for** $i = 0$ to $2^{\bar{k}} - 1$ **do**
- 8: **for** $j = 0$ to $2^{k-\bar{k}} - 1$ **do**
- 9: $\llbracket s_{i+2^{\bar{k}} \cdot j} \rrbracket \leftarrow \text{Mul}(\llbracket b_i \rrbracket, \llbracket c_j \rrbracket)$
- 10: **end for**
- 11: **end for**
- 12: **return** $\llbracket s_0, \dots, s_{2^k-1} \rrbracket$

Hence, our protocol for generating random one-hot vector of length t (denoted as $\text{RandOneHot}(t)$) works as follows:

- 1) On (public) input t , the parties fetch $\log \bar{t}$ random bits $\llbracket r_0 \rrbracket, \dots, \llbracket r_{\log \bar{t}-1} \rrbracket$.
- 2) Each party computes locally its share for $\sum_{i=0}^{\log \bar{t}-1} 2^i \llbracket r_i \rrbracket$, which is the secret index $\llbracket r \rrbracket$.

- 3) The parties run Demux protocol to generate the random one-hot vector with secret shared index $\llbracket r \rrbracket$.

Please see algorithm 2 for a formal description of the RandOneHot protocol.

Without loss of generality, we assume that t is a power of 2. The $\text{RandOneHot}(t)$ protocol requires $< 2t$ secure multiplications, $\log \log t$ rounds. It incurs $O(tm)$ computation and communication in the offline phase, where each atomic operation is treated as $O(1)$, since our protocol does not depend on a specific secret-sharing implementation.

Algorithm 2 $(\llbracket \bar{s} \rrbracket, \llbracket r \rrbracket) \leftarrow \text{RandOneHot}(t)$

Ensure: $\llbracket \bar{s} \rrbracket$ is a one-hot vector of length $\bar{t}$, with index $\llbracket r \rrbracket$.

- 1: Let $\bar{t} \leftarrow 2^{\lceil \log t \rceil}$
 - 2: **for** $i = 0$ to $\log \bar{t} - 1$ **do**
 - 3: $\llbracket r_i \rrbracket \leftarrow \text{RandBit}()$
 - 4: **end for**
 - 5: $\llbracket r \rrbracket \leftarrow \sum_{i=0}^{\log \bar{t}-1} 2^i \cdot \llbracket r_i \rrbracket$
 - 6: $\llbracket \bar{s} \rrbracket \leftarrow \text{Demux}(\llbracket r_0 \rrbracket, \dots, \llbracket r_{\log \bar{t}-1} \rrbracket)$
 - 7: **return** $(\llbracket \bar{s} \rrbracket, \llbracket r \rrbracket)$
-

B. One-hot Encoding

Our target is to generate a one-hot vector for a secret shared index $\llbracket k \rrbracket$, with the help of the random one-hot vector $\llbracket \bar{s} \rrbracket$ and its index $\llbracket r \rrbracket$. The intuition is, it is safe to open $k - r \pmod{\bar{t}}$ to all parties, which is uniformly distributed over $[\bar{t}-1]$. With this value, which is the deviation from the random one-hot vector to the desired one-hot vector, each party can shift locally the hot bit to the correct entry.

Our construction works almost the same as this intuition, except that instead of doing modulo operation in secret shared form, we do it in plaintext. The reason is that modulo operation is costly and requires another $\log \bar{t}$ fresh random bits[8], while computing modulo for a plaintext element is almost for free.

Recall that $k - r \in [-(\bar{t}-1), t-1]$ is not uniformly distributed, and thus directly opening it is unsafe. Hence, we add to it a random “sufficiently large” mask that is divisible by $\bar{t}$, and then open it. This mask is produced by the RandSum protocol, and note that since (see section III)

$$\log p > \lceil \log t \rceil + \lceil \log m \rceil + \kappa,$$

the sum does not overflow modulo p . We will prove later that this is statistically secure w.r.t. κ .

See algorithm 3 for the formal description of the protocol.

The $\text{Encode}(t, \llbracket k \rrbracket)$ protocol requires, in the offline phase, $< 2t$ secure multiplications and $\log \log t$ communication rounds, resulting in $O(tm)$ computational and communication costs. In addition, one communication round with $O(m)$ communication and $O(tm)$ computation is incurred in either the online or offline phase, depending on whether $\llbracket k \rrbracket$ is generated online.

C. One-hot Encoding in Constant Rounds

Below we show how to generate a one-hot vector in constant rounds without random one-hot vector. Nevertheless, note that

Algorithm 3 ($\llbracket \bar{s} \rrbracket$) $\leftarrow$ Encode($t, \llbracket k \rrbracket$)**Require:** $k \in [t-1]$.**Ensure:** $\llbracket \bar{s} \rrbracket$ is a one-hot vector of length t , with index $\llbracket k \rrbracket$.

```

1: Let  $\bar{t} \leftarrow 2^{\lceil \log t \rceil}$ 
2:  $(\llbracket \bar{s}' \rrbracket, \llbracket r \rrbracket) \leftarrow \text{RandOneHot}(t)$ 
3:  $\llbracket d \rrbracket \leftarrow \text{RandSum}()$ 
4:  $\llbracket d' \rrbracket \leftarrow \llbracket k \rrbracket - \llbracket r \rrbracket + \bar{t} + \bar{t} \cdot \llbracket d \rrbracket$ 
5:  $\delta' \leftarrow \text{Open}(\llbracket d' \rrbracket)$ 
6:  $\delta \leftarrow \delta' \bmod \bar{t}$ 
7: for  $j = 0$  to  $\bar{t} - 1$  do
8:   Each  $P_i$  let  $\llbracket s_j \rrbracket_i \leftarrow \llbracket s'_{j+\bar{t}-\delta} \rrbracket_i$ 
9: end for
10: return  $\llbracket \bar{s} \rrbracket = \llbracket s_0, \dots, s_{t-1} \rrbracket$   $\triangleright$  truncate to length  $t$ 

```

since term $\log \log t$ is “almost constant”, and the constant-round protocol requires more secret random resources, it is possible that the protocol in former subsection performs better in practice. Also, generating random one-hot vectors offline helps reduce online communication, which is often more important in MPC practice.

Denote the protocol to convert an integer to its one-hot vector in constant rounds by EncodeC. It relies on a technique to evaluate an arbitrary publicly known polynomial at a secret shared point in constant rounds, which in turn utilizes a constant rounds prefix multiplication protocol proposed in [8]. According to [8], the idea of this prefix multiplication protocol, PreMulC, originates from [44]. To formalize,

$$(\llbracket p_0 \rrbracket, \llbracket p_1 \rrbracket, \dots, \llbracket p_{t-1} \rrbracket) \leftarrow \text{PreMulC}(\llbracket x_0 \rrbracket, \llbracket x_1 \rrbracket, \dots, \llbracket x_{t-1} \rrbracket),$$

where $p_i = \prod_{j=0}^i x_j$ for all $i \in [t-1]$. This protocol requires that all x_i are non-zero, and involves $2t$ multiplications in total [8]. It is straightforward to see that this protocol can help raise x to arbitrary powers in constant rounds, by simply feeding the protocol with repeated $\llbracket x \rrbracket$.

Now for $i \in [1, t]$, let F_i be the polynomial of degree $t-1$, with $F_i(i) = 1$ and $F_i(j) = 0$ for all $j \in [1, t] \setminus \{i\}$. This polynomial is unique and can be computed via Lagrange interpolation. For any shared $x \in [1, t]$, by acquiring $\llbracket x^j \rrbracket$ for $j \in [1, t-1]$ via PreMulC, parties can now locally evaluate $F_i(\llbracket x \rrbracket)$ for all $i \in [1, t]$, as the coefficients of F_i are all public. Hence, by evaluating $\llbracket s_{i-1} \rrbracket \leftarrow F_i(\llbracket x \rrbracket)$, we obtain the one-hot vector with index $\llbracket x-1 \rrbracket$: only the polynomial F_i with $i = x$ will be evaluated to 1.

We denote this protocol as EncodeC. This protocol is formally described in algorithm 4.

The EncodeC($t, \llbracket k \rrbracket$) protocol requires $2(t-1)$ secure multiplications, with $O(t^2m)$ computational, $O(tm)$ communication costs and 3 communication rounds. These costs occur in either the online or offline phase, depending on whether $\llbracket k \rrbracket$ is generated online. Strictly, computing and distributing F_i requires $O(t)$ computation and $O(tm)$ communication in offline phase.

Note that the protocol EncodeC can also be used to generate a random one-hot vector of index in $[t-1]$, with $\lceil \log t \rceil$ random bits and a call to EncodeC($\bar{t}, \sum_i 2^i \llbracket r_i \rrbracket$). Hence, the protocol to generate a random one-hot vector also has two versions,

Algorithm 4 ($\llbracket \bar{s} \rrbracket$) $\leftarrow$ EncodeC($t, \llbracket k \rrbracket$)**Require:** For each $i \in [1, t]$, F_i is the unique $t-1$ degree polynomial such that $F_i(i) = 1$ and $F_i(j) = 0$ for $j \in [1, t] \setminus \{i\}$.**Ensure:** $\llbracket \bar{s} \rrbracket$ is a one-hot vector of length t , with index $\llbracket k \rrbracket$.

```

1:  $\llbracket \bar{p} \rrbracket \leftarrow \text{PreMulC}(\llbracket k+1 \rrbracket, \llbracket k+1 \rrbracket, \dots, \llbracket k+1 \rrbracket) \triangleright t-1$ 
   many
2: for  $i = 1$  to  $t$  do parallel
3:   Assume  $F_i(x) = \sum_{j=0}^{t-1} F_{i,j} x^j$ 
4:    $\llbracket s_{i-1} \rrbracket \leftarrow F_{i,0} + \sum_{j=1}^{t-1} F_{i,j} \llbracket p_{j-1} \rrbracket$ 
5: end for
6: return  $\llbracket s_0, \dots, s_{t-1} \rrbracket$ 

```

one in $\log \log t$ rounds and one in constant rounds.

D. Security Analysis

The security of protocol Demux follows directly from the security of the underlying multiplication protocol. The security of protocol RandOneHot relies on the uniformity of the bit generated by RandBit, together with the security of Demux. For protocol EncodeC, its security reduces to that of the underlying protocol PreMulC.

Since the security of these three protocols can be straightforwardly reduced to the security of their respective underlying primitives, we omit their formal security proofs and only state the result. It remains to establish the security of protocol Encode.

We prove this using a hybrid argument. First, we consider an idealized version of protocol Encode, in which the opened deviation value is uniformly distributed over $\llbracket \bar{t} \rrbracket$ and independent of the target index k . We then show that the output distribution of this ideal protocol is statistically indistinguishable from that of the real protocol Encode, with respect to the security parameter κ .

Lemma 1. *Assume that all parties jointly hold a secret random one-hot vector $\llbracket \bar{s}' \rrbracket$ and its index $\llbracket r \rrbracket$, generated by calling protocol RandOneHot. Consider the following ideal functionality $\mathcal{F}$:*

- 1) Each party P_i sends its shares $\llbracket k \rrbracket_i$ and $\llbracket r \rrbracket_i$ to $\mathcal{F}$.
- 2) $\mathcal{F}$ reconstructs k and r .
- 3) $\mathcal{F}$ samples $d_i \xleftarrow{\$} [2^\kappa]$ independently for each $i \in [t+1]$.
- 4) $\mathcal{F}$ sends

$$\delta'_\mathcal{F} := (\bar{t} + k - r \bmod \bar{t}) + \bar{t} \sum_{i \in [t+1]} d_i$$

to all parties.

- 5) Each party P_i computes $\delta_\mathcal{F} := \delta'_\mathcal{F} \bmod \bar{t}$, and shifts the random one-hot vector in the same manner as in protocol Encode, namely,

$$\llbracket s_j \rrbracket_i \leftarrow \llbracket s'_{j+\bar{t}-\delta_\mathcal{F}} \rrbracket_i.$$

Then $\mathcal{F}$ realizes the functionality of protocol Encode with information-theoretic security.

Proof: [Sketch] It is immediate that $\mathcal{F}$ implements the same (correct) functionality as protocol Encode.

To see information-theoretic security, observe that the term

$$\bar{t} + k - r \bmod \bar{t}$$

is uniformly distributed over $[\bar{t}]$, independently of k . Indeed, since the index r is uniformly random and never revealed to any party (except the trusted ideal functionality), the resulting value modulo $\bar{t}$ is uniform from the perspective of any adversarial coalition.

We remark that the ideal functionality could simply send the term

$$\bar{t} + k - r \bmod \bar{t}$$

to all parties while still achieving information-theoretic security. However, for analytical purposes that will become clear shortly, we let $\mathcal{F}$ include an additional random mask, mirroring the behavior of protocol Encode.

We now show that $\delta'_{\mathcal{F}}$ is statistically close to the value δ' opened in protocol Encode. Since δ' is the only value revealed during the protocol, statistical indistinguishability of these two distributions implies the statistical security of Encode.

Below we present two lemmas. In the following, Δ denotes statistical distance and all additions are taken modulo p .

Lemma 2. *Let X, Y, Z be random variables over $\mathbb{Z}_p$, where Z is independent of both X and Y . Then*

$$\Delta(X + Z; Y + Z) \leq \Delta(X; Y).$$

Lemma 3. *Let X, Y, Z be random variables over $\mathbb{Z}_p$, where Z is independent of both X and Y . Then*

$$\Delta(X + Z, Z; Y + Z, Z) = \Delta(X; Y).$$

The proofs of the above lemmas are omitted due to space constraints. We are now ready to establish the security of the protocol, which follows as a direct corollary of the next theorem.

Theorem 1. *For any adversarial coalition of size at most t , the distributions of $\delta'_{\mathcal{F}}$ and δ' satisfy*

$$\Delta(\delta'_{\mathcal{F}}; \delta') \leq \frac{1}{2^\kappa}.$$

Proof: Recall that protocol RandSum involves m parties, indexed by a set M , each contributing a random value in $[2^\kappa]$. Let $M' \subseteq M$ denote the subset of honest parties, which is non-empty.

Let k be the target index, and let X be uniformly distributed over $[\bar{t} - 1]$. For each $i \in M$, let Y_i denote the random value chosen by party P_i . Finally, let Z be uniformly distributed over $[\bar{t}2^\kappa]$.

It suffices to consider the view of the adversarial coalition, namely to prove $\Delta(\delta'_{\mathcal{F}}, (Y_i)_{i \in M \setminus M'}; \delta', (Y_i)_{i \in M \setminus M'}) \leq \frac{1}{2^\kappa}$. By lemma 3, it suffices to prove $\Delta(\delta'_{\mathcal{F}} - \bar{t} \sum_{i \in M \setminus M'} Y_i; \delta' - \bar{t} \sum_{i \in M \setminus M'} Y_i) \leq \frac{1}{2^\kappa}$.

It follows that

$$\begin{aligned} & \Delta\left(\delta'_{\mathcal{F}} - \bar{t} \sum_{i \in M \setminus M'} Y_i; \delta' - \bar{t} \sum_{i \in M \setminus M'} Y_i\right) \\ &= \Delta\left(k + X + \bar{t} \sum_{i \in M'} Y_i; X + \bar{t} + \bar{t} \sum_{i \in M'} Y_i\right) \\ &\leq \Delta\left(k + X + \bar{t} Y_i; X + \bar{t} + \bar{t} Y_i\right) \quad \text{for any } i \in M' \\ &= \Delta(k + Z; \bar{t} + Z) \leq \frac{\bar{t} - k}{\bar{t}2^\kappa} \leq \frac{1}{2^\kappa}, \end{aligned}$$

where we used lemma 2 and the fact that $X + \bar{t} Y_i$ is uniformly distributed over $[\bar{t}2^\kappa]$.

By Theorem theorem 1, the security of protocol Encode can be formalized as follows.

Theorem 2. *Protocol Encode is statistically secure, in the sense that it is statistically indistinguishable from the ideal functionality described in lemma 1, with respect to the security parameter κ .*

V. COMPARISON AND DIGIT DECOMPOSITION

In this section, we construct an efficient and secure digit decomposition protocol supporting an arbitrary integer base t . This protocol decomposes a secret-shared integer $\llbracket x \rrbracket$ into $\llbracket x_0 \rrbracket, \llbracket x_1 \rrbracket, \dots, \llbracket x_{k-1} \rrbracket$ in its base- t representation, that is:

$$\llbracket x \rrbracket = \sum_{i=0}^{k-1} \llbracket x_i \rrbracket \cdot t^i,$$

where $0 \leq x_i < t$. This is a generalization of bit decomposition [23] and is useful in various applications including data processing and machine learning [26, 27, 28].

As discussed in Section II, similar to the case of bit decomposition, to compute the digit decomposition of $\llbracket x \rrbracket$ in base- t where $0 \leq x < t^k$, we first generate a random digit $\llbracket a \rrbracket$ with its base- t digits $\llbracket a_i \rrbracket_{i \in [k]}$. We then compute

$$\llbracket y \rrbracket = \llbracket x \rrbracket + \llbracket a \rrbracket + t^k \cdot \llbracket r \rrbracket$$

and open it to all parties, with $\llbracket r \rrbracket \leftarrow \text{RandSum}()$. After decomposing the plaintext value $(y \bmod t^k)$ into base- t digits $\llbracket y_i \rrbracket_{i \in [k]}$, we can obtain the base- t digits of $\llbracket x \rrbracket$ by computing

$$\llbracket x_i \rrbracket = \text{Sub}(t, \llbracket y_i \rrbracket_{i \in [k]}, \llbracket a_i \rrbracket_{i \in [k]}),$$

where $\text{Sub}(t, \cdot, \cdot)$ performs subtraction in base- t .

Hence, in this section, we will

- 1) construct the random digit generation protocol $\text{RandDigit}(t)$, for generating a random digit $\llbracket a \rrbracket$ with base- t digits $\llbracket a_i \rrbracket_{i \in [k]}$.
- 2) construct the constant-round subtraction protocol SubC , which requires an efficient comparison protocol as a subroutine.

A. Efficient Comparison via One-Hot Encoding

In MPC, comparison operations are typically expensive. Existing comparison protocols consider generic comparisons between two secret-shared integers, which are often based on the comparison of their binary representations [14, 15, 16]. However, in our setting, parties already know the range of the

secret $\llbracket x \rrbracket$ and $\llbracket y \rrbracket$ to be compared, i.e. $x, y \in [t]$. This enables the following simple yet efficient comparison protocol.

Recall that one-hot encoding represents an integer $x \in [t-1]$ as a vector $\mathbf{u}$, where the x -th element is 1 and the rest are 0, i.e.,

$$\vec{u} = (\underbrace{0, 0, \dots, 0}_x, 1, 0, \dots, 0)$$

Clearly, for any integer $c \in [t-1]$, checking whether $x < c$ is equivalent to checking whether the sum of the first c entries equals 1.

$\text{Cmp}_{<}(c, t, \llbracket x \rrbracket)$ checks if a secret-shared integer x is less than a public constant c :

$$\text{Cmp}_{<}(c, t, \llbracket x \rrbracket) = \begin{cases} 1 & \text{if } x < c \\ 0 & \text{otherwise} \end{cases}$$

The protocol converts x to a one-hot vector, which can be done with protocol `Encode` or `EncodeC`, then sums the first c elements to determine if $x < c$. See algorithm 5 for the protocol's formal description.

Algorithm 5 $\llbracket s \rrbracket \leftarrow \text{Cmp}_{<}(c, t, \llbracket x \rrbracket)$

Require: Secret $x \in [t-1]$

Ensure: $\llbracket s \rrbracket = 1$ if $x < c$, 0 otherwise

- 1: $\llbracket \vec{u} \rrbracket \leftarrow \text{EncodeC}(t, \llbracket x \rrbracket)$
 - 2: **return** $\llbracket s \rrbracket \leftarrow \sum_{i=0}^{c-1} \llbracket u_i \rrbracket$
-

Similarly, $\text{Cmp}_{=}(c, t, \llbracket x \rrbracket)$ checks if a secret-shared integer equals a public constant:

$$\text{Cmp}_{=}(c, t, \llbracket x \rrbracket) = \begin{cases} 1 & \text{if } x = c \\ 0 & \text{otherwise} \end{cases}$$

Using the same one-hot encoding, the protocol accesses the c -th element of the encoded vector to determine equality. See algorithm 6 for the protocol's formal description.

Algorithm 6 $\llbracket s \rrbracket \leftarrow \text{Cmp}_{=}(c, t, \llbracket x \rrbracket)$

Require: Secret $x \in [t-1]$

Ensure: $\llbracket s \rrbracket = 1$ if $x = c$, 0 otherwise

- 1: $\llbracket \vec{u} \rrbracket \leftarrow \text{EncodeC}(t, \llbracket x \rrbracket)$
 - 2: **return** $\llbracket s \rrbracket \leftarrow \llbracket u_c \rrbracket$
-

The complexities of $\text{Cmp}_{=}$ and $\text{Cmp}_{<}$ are identical to those of their corresponding encoding protocols, `EncodeC` or `Encode`, respectively.

We note that in practice, when three-way comparison is needed, instead of separately calling $\text{Cmp}_{=}$ and $\text{Cmp}_{<}$, we can reuse the one-hot encoding of $\llbracket x \rrbracket$ to perform both comparisons simultaneously.

B. Random Digit Generation

The $\text{RandDigit}(t)$ primitive generates a secret-shared random digit $\llbracket r \rrbracket$ uniformly distributed within the range $[t-1]$. This primitive employs a rejection sampling technique to ensure that the generated random number falls within the specified range while preserving security properties.

Let $l := \lceil \log t \rceil$. Parties first generate l random bits $(\llbracket b_i \rrbracket)_{i \in [l]}$ using the `RandBit` protocol and combine them by

$$\llbracket b \rrbracket \leftarrow \sum_{i=0}^{l-1} 2^i \cdot \llbracket b_i \rrbracket.$$

Parties then check whether b is less than t by the $\text{Cmp}_{<}(t, t, \llbracket b \rrbracket)$ protocol. If $b < t$, the value is accepted; otherwise, the process is repeated until a valid number is generated. Since b is uniform in $[2^l - 1]$, the probability of acceptance is at least $1/2$, which makes the protocol efficient in expectation. It is also clear that the generated number is secret and uniform in $[t-1]$.

The formal description of this protocol can be found in algorithm 7. In practice, since the amount of randomness required is typically large, parties can generate multiple random digits in parallel and in advance to amortize the cost of communication rounds.

Algorithm 7 $\llbracket r \rrbracket \leftarrow \text{RandDigit}(t)$

Ensure: Return $\llbracket r \rrbracket$ with r sampled uniformly from $[t-1]$

- 1: $l \leftarrow \lceil \log t \rceil$
 - 2: $t' \leftarrow 2^l$
 - 3: **repeat**
 - 4: $\llbracket b_0, \dots, b_{l-1} \rrbracket \leftarrow \text{RandBit}()$
 - 5: $\llbracket b \rrbracket \leftarrow \sum_{i=0}^{l-1} \llbracket b_i \rrbracket \cdot 2^i$
 - 6: $\llbracket s \rrbracket \leftarrow \text{Cmp}_{<}(t, t', \llbracket b \rrbracket)$
 - 7: $s \leftarrow \text{Open}(\llbracket s \rrbracket)$
 - 8: **until** $s = 1$
 - 9: **return** $\llbracket r \rrbracket \leftarrow \llbracket b \rrbracket$
-

C. Subtraction in Constant Rounds

Now we are ready to perform efficient subtraction operation that computes the difference between two integers represented in base- t . Given $\llbracket a_i \rrbracket_{i \in [k]}$ and $\llbracket b_i \rrbracket_{i \in [k]}$ with each element in $[t-1]$, let $a = \sum_{i=0}^{k-1} a_i t^i$ and $b = \sum_{i=0}^{k-1} b_i t^i$. The protocol computes

$$\llbracket c_i \rrbracket_{i \in [k]} \leftarrow \text{SubC}(t, \llbracket a_i \rrbracket_{i \in [k]}, \llbracket b_i \rrbracket_{i \in [k]}),$$

where $c_i \in [t-1]$ and

$$c \equiv a - b \equiv \sum_{i=0}^{k-1} c_i t^i \pmod{t^k}.$$

Note that the subtraction is performed under modulo t^k , which means in the case of $a < b$, $c = a - b + t^k$.

As a starting point, consider the following naive construction. The parties first compute $\llbracket s_0 \rrbracket$ by

$$\llbracket s_0 \rrbracket \leftarrow \text{Cmp}_{<}(t, 2 \cdot t, t + \llbracket a_0 \rrbracket - \llbracket b_0 \rrbracket),$$

which is 1 if $a_0 < b_0$ and 0 otherwise. s_0 indicates whether a borrow is needed for the least significant digit. Similarly, for each digit position i , the parties compute $\llbracket s_i \rrbracket$ by

$$\llbracket s_i \rrbracket \leftarrow \text{Cmp}_{<}(t, 2 \cdot t, t + \llbracket a_i \rrbracket - \llbracket b_i \rrbracket - \llbracket s_{i-1} \rrbracket),$$

which is 1 if $a_i < b_i + s_{i-1}$ and 0 otherwise. After obtaining all borrow bits $\llbracket s_i \rrbracket$, the parties can compute each result digit $\llbracket c_i \rrbracket$ as

$$\llbracket c_i \rrbracket \leftarrow \llbracket a_i \rrbracket - \llbracket b_i \rrbracket + t \cdot \llbracket s_i \rrbracket.$$

The problem now is how to compute all borrow bits $\llbracket s_i \rrbracket$ efficiently. While the above approach works, it incurs non-constant $O(k)$ communication rounds due to the sequential dependency of borrow bits.

Although t -wise subtraction has received limited attention in the MPC literature, we observe that the borrow propagation in subtraction shares similar properties with carry propagation in bit-wise addition, which is well-studied in MPC. Consider temporarily the case of bit-wise addition, where two k -bit integers x and y are represented by x_i and y_i respectively. In the propagation of bit-wise addition, a carry bit t_i is decided by the following rule:

$$t_i = \begin{cases} 1 & \text{if } x_i + y_i = 2, \\ 0 & \text{if } x_i + y_i = 0, \\ t_{i-1} & \text{if } x_i + y_i = 1. \end{cases}$$

In literature, the protocols that compute above carry bits t_i are already well studied [9, 23]. To leverage existing results, we denote the constant-round carry propagation protocol as

$$\llbracket t_i \rrbracket_{i \in [k]} \leftarrow \text{CarryC}(\llbracket x_i \rrbracket_{i \in [k]}, \llbracket y_i \rrbracket_{i \in [k]}).$$

In our case of subtraction between a and b , the borrow bit s_i follows a similar rule:

$$s_i = \begin{cases} 1 & \text{if } a_i < b_i, \\ 0 & \text{if } a_i > b_i, \\ s_{i-1} & \text{if } a_i = b_i. \end{cases}$$

It is hence clear how we can reduce the problem of borrow propagation to the problem of carry propagation, and utilize the same constant-round protocol. In our constant round subtraction protocol, we first compute comparison bits for each digit position i , which converts task of computing borrow bits to computing carry bits. After obtaining all borrow bits from CarryC protocol, parties can compute each result digit $\llbracket c_i \rrbracket$ easily.

In algorithm 8 we present the fully developed constant-round base- t subtraction protocol SubC.

D. Digit Decomposition for Generic Base- t

Building upon the previously established primitives for comparison, random digit generation, and constant-round subtraction, we now present a constant-round decomposition protocol DigitDecomp($t, \llbracket x \rrbracket$). This protocol efficiently decomposes a secret-shared integer $\llbracket x \rrbracket$ of bit-length L into its base- t representation. The decomposition produces $k = \lceil \frac{L}{\log t} \rceil$ digits. As mentioned, parties first generate a random digit $\llbracket a \rrbracket$ with its base- t digits $\llbracket a_i \rrbracket_{i \in [k]}$ using the RandDigit(t) protocol. Parties fetch an additional random $\llbracket r \rrbracket \leftarrow \text{RandSum}()$, and open the sum

$$\llbracket d \rrbracket \leftarrow \llbracket x \rrbracket + \llbracket a \rrbracket + t^k \cdot \llbracket r \rrbracket.$$

Note that since $\llbracket r \rrbracket$ is shifted by t^k , the lowest k digits of d in base- t are not affected by $\llbracket r \rrbracket$. After computing the subtraction

Algorithm 8 $\llbracket \vec{c} \rrbracket \leftarrow \text{SubC}(t, \llbracket \vec{a} \rrbracket, \llbracket \vec{b} \rrbracket)$

Require: $\llbracket \vec{a} \rrbracket = \llbracket a_0, a_1, \dots, a_{k-1} \rrbracket$, $\llbracket \vec{b} \rrbracket = \llbracket b_0, b_1, \dots, b_{k-1} \rrbracket$
 with $0 \leq a_i, b_i < t$
Ensure: $\sum_i c_i \cdot t^i = \sum_i a_i \cdot t^i - \sum_i b_i \cdot t^i \pmod{t^k}$, with $0 \leq c_i < t$

- 1: **for** $i = 0$ to $k - 1$ **do**
- 2: $\llbracket d_i \rrbracket \leftarrow t + \llbracket a_i \rrbracket - \llbracket b_i \rrbracket$
- 3: $\llbracket x_i \rrbracket \leftarrow \text{Cmp}_{<}(t, 2t, \llbracket d_i \rrbracket)$
- 4: $\llbracket y_i \rrbracket \leftarrow \llbracket x_i \rrbracket + \text{Cmp}_{=}(t, 2t, \llbracket d_i \rrbracket)$
- 5: **end for**
- 6: $\llbracket \vec{s} \rrbracket \leftarrow \text{CarryC}(\llbracket \vec{x} \rrbracket, \llbracket \vec{y} \rrbracket)$
- 7: $\llbracket c_0 \rrbracket \leftarrow t \cdot \llbracket s_0 \rrbracket + \llbracket a_0 \rrbracket - \llbracket b_0 \rrbracket$
- 8: **for** $i = 1$ to $k - 1$ **do**
- 9: $\llbracket c_i \rrbracket \leftarrow t \cdot \llbracket s_i \rrbracket - \llbracket s_{i-1} \rrbracket + \llbracket a_i \rrbracket - \llbracket b_i \rrbracket$
- 10: **end for**
- 11: **return** $\llbracket \vec{c} \rrbracket = (\llbracket c_0 \rrbracket, \dots, \llbracket c_{k-1} \rrbracket)$

between the lowest k digits of d and $\llbracket a_i \rrbracket_{i \in [k]}$ using the SubC protocol, the result is correctly base- t -decomposed $\llbracket x \rrbracket$.

For the security of the protocol, since $\llbracket a \rrbracket$ is uniform in $[t^k - 1]$ and $\llbracket r \rrbracket$ is a summation of uniform $\llbracket r_i \rrbracket$ in $[2^\kappa]$, it should be clear that, following a similar argument as theorem 1,

$$\Delta(x + a + t^k \cdot r; a + t^k \cdot r) = O(2^{-\kappa}).$$

Hence, $\llbracket d \rrbracket$ is almost independent of $\llbracket x \rrbracket$, ensuring the security of the protocol.

The protocol is formally presented in algorithm 9.

Algorithm 9 $\llbracket \vec{c} \rrbracket \leftarrow \text{DigitDecomp}(t, \llbracket x \rrbracket)$

Require: Secret $x < t^k$
Ensure: Decompose x into t -based digits $\llbracket c_i \rrbracket$ s.t. $x = \sum_i c_i t^i$

- 1: $\llbracket a_i \rrbracket \leftarrow \text{RandDigit}(t)$ for $i \in [k]$
- 2: $\llbracket r \rrbracket \leftarrow \text{RandSum}()$
- 3: $\llbracket a \rrbracket \leftarrow t^k \cdot \llbracket r \rrbracket + \sum_{i=0}^{k-1} \llbracket a_i \rrbracket \cdot t^i$
- 4: **Open** $\llbracket d \rrbracket \leftarrow \llbracket x \rrbracket + \llbracket a \rrbracket$
- 5: Let $(d_i)_{i=0}^{k-1}$ be t -based digits representation of $d \pmod{t^k}$
- 6: $\llbracket \vec{c} \rrbracket \leftarrow \text{SubC}(t, (d_i)_{i=0}^{k-1}, \llbracket a_i \rrbracket_{i=0}^{k-1})$
- 7: **return** $\llbracket \vec{c} \rrbracket$

Here, we recall $k = \lceil \frac{L}{\log t} \rceil$. The DigitDecomp($t, \llbracket x \rrbracket$) protocol incurs $O(ktm)$ online communication. With Encode, online computation is $O(ktm)$, in $O(\log t)$ rounds, with $4k$ secure multiplications; with EncodeC, computation is $O(kt^2m)$, in $O(1)$ rounds, with $(4t+2)k$ secure multiplications. Offline, costs are $O(ktm)$, $O(1)$ rounds, with $< 6tk$ (Encode) or $(2t-2)k$ (EncodeC) multiplications.

VI. RADIX SORT

In this section, we consider a protocol among m parties with n inputs. We first focus on vectors of length L , where each entry is a small integer in the range $[t-1]$, so that one-hot encoding is applicable. We then extend the radix-sort protocol to support general integers. The cases of vectors over general integers and key-value sorting follow straightforwardly from these constructions.

A. Vector Radix Sort

Assume the inputs are n secret shared vectors $\llbracket \vec{v}_0 \rrbracket, \llbracket \vec{v}_1 \rrbracket, \dots, \llbracket \vec{v}_{n-1} \rrbracket$, with each vector of length L and each entry $v_{i,j}$ in $[t-1]$. Denote $V_i \in \mathbb{Z}$ as their integer representation, i.e.

$$V_i := \sum_{j=0}^{L-1} t^j v_{i,j}.$$

Note that V_i may exceed the field modulus p .

The radix sort protocol reorders the input vectors by a permutation $\pi : [n-1] \rightarrow [n-1]$, such that

$$\forall i \in [0, n-2] : V_{\pi(i)} \leq V_{\pi(i+1)}.$$

The output of the protocol is the reordered vectors, i.e.

$$\llbracket \vec{v}_{\pi(0)} \rrbracket, \llbracket \vec{v}_{\pi(1)} \rrbracket, \dots, \llbracket \vec{v}_{\pi(n-1)} \rrbracket.$$

Let's denote the vector radix sort protocol we are to construct as **VecRadixSort**.

To implement radix sort for vectors, we follow the approach of Hamada et al. [20], which relies on two building blocks: the rank-computation protocol **RankComp** and the reveal-then-sort protocol **RevealSort**. **RankComp** takes n secret-shared keys, implicitly performs a stable sort, and outputs the corresponding rank vector $\vec{r}$. **RevealSort** takes the secret-shared rank vector $\vec{r}$ and values $\llbracket \vec{v}_i \rrbracket_{i \in [n]}$, and outputs them reordered according to $\vec{r}$. Together, these two sub-protocols implement the core step of radix sort: in each round, the vectors are ranked by the current digit and then reordered for the next round.

We note that the original rank-computation protocol of Hamada et al. [20] supports only binary inputs, restricting radix sort to base $t = 2$ (see Algorithm 1 in [20]). In this work, we generalize their protocol to support an arbitrary base t . For a suitable choice of t , this reduces the number of calls to **RevealSort**—typically the main bottleneck—thereby improving communication, computation, and round complexity.

RankComp Protocol The **RankComp** protocol takes as input n secret-shared keys $\llbracket k_0 \rrbracket, \dots, \llbracket k_{n-1} \rrbracket$ and outputs a secret-shared rank vector $\llbracket \vec{r} \rrbracket = (\llbracket r_0 \rrbracket, \dots, \llbracket r_{n-1} \rrbracket)$, which forms a permutation over $[n-1]$. The protocol is stable: if $k_i = k_j$ and $i < j$, then $r_i < r_j$.

The keys are first transformed into one-hot vectors of length t using **Encode** or **EncodeC**, yielding a matrix $\llbracket A \rrbracket = (\llbracket a_{i,j} \rrbracket)_{i \in [n], j \in [t]}$, where each row is a one-hot vector.

Parties then compute the prefix sums

$$\llbracket c_{i,j} \rrbracket \leftarrow \sum_{\substack{x \in [n] \\ y < j}} \llbracket a_{x,y} \rrbracket + \sum_{x \leq i} \llbracket a_{x,j} \rrbracket,$$

for all $i \in [n]$ and $j \in [t]$. For $k_i = j$, the value $\llbracket c_{i,j} \rrbracket - 1$ equals the stable rank of k_i . The rank is therefore obtained by selecting the corresponding entry via $\llbracket r_i \rrbracket \leftarrow \llbracket \vec{a}_i \rrbracket \cdot \llbracket \vec{c}_i \rrbracket - 1$.

All computations are performed over secret-shared data, and no intermediate values are opened. Security follows directly from that of the underlying secret-sharing scheme and its

primitives.

RevealSort Protocol This protocol is due to Hamada et al. [20]; for completeness, we briefly review it.

The protocol takes as input a list of secret-shared key-value pairs, where each value is a vector of fixed length and, in our setting, the corresponding key is its rank produced by **RankComp**. It outputs the pairs sorted according to the keys.

The parties first apply a random shuffle to all key-value pairs, then open the shuffled keys and sort accordingly. Since the permutation is randomly chosen and kept secret, the opened keys cannot be linked to the original inputs. Hence, revealing the shuffled keys does not disclose the rank of any specific input vector.

VecRadixSort Protocol Using **RankComp** and **RevealSort** as building blocks, we now construct the vector radix sort protocol.

We first extend **RankComp** to vectors, obtaining the protocol **VecRankComp**. The protocol takes as input integers n, t and secret-shared vectors $\llbracket \vec{v}_i \rrbracket_{i \in [n]}$, where each entry $v_{i,j}$ lies in $[t-1]$. It outputs the rank of each vector, thereby essentially completing the radix sort over the vectors.

Instead of permuting entire vectors in each round, the protocol reorders only the next digit column, proceeding from the LSB to the MSB. It maintains a value-index pair $(\vec{v}', \vec{s})$, where $\vec{v}'$ contains the current digit and $\vec{s}$ records the corresponding original indices. In the j -th round, the protocol computes the rank vector $\vec{r}$ of $\vec{v}'$ using **RankComp**. It then invokes **RevealSort** twice: first to reorder $\vec{r}$ according to the index vector $\vec{s}$, and then to sort the pairs $(v_{i,j+1}, s_i)_{i \in [n]}$ according to the reordered ranks, thereby preparing the next round. After processing all digits, the final ranks are reordered according to the original indices and returned.

The vector radix sort protocol **VecRadixSort** is obtained directly from **VecRankComp**. Given the input vectors, it computes their ranks via **VecRankComp** and subsequently reorders them using **RevealSort**.

B. Integer Radix Sort

To sort integers, one may directly apply **RankComp** and **RevealSort**. However, for large integers, this would require extremely long one-hot vectors.

A more efficient approach is to first decompose each integer into base- t digits using **DigitDecomp**, and then apply **VecRadixSort**. This limits the one-hot encoding length to t .

Given a publicly agreed base t , algorithm 10 presents the complete integer sorting protocol. The choice of t provides a trade-off: a larger t reduces the number of digits and thus the number of rounds and shuffle invocations, while increasing the per-round communication cost due to longer one-hot vectors. We further discuss the protocol complexity in section VII-A.

VII. EVALUATION

We conduct a comprehensive evaluation of our work, encompassing both theoretical and experimental analyses. In this section, we first provide a complexity analysis of our protocols, then present an experimental study highlighting the performance advantages of **UFO_s** over existing constructions.

TABLE I: Secure-operation complexity of the proposed protocols

Protocol	Mul	RandSum	RandBit
Encode	$< 2t$	1	$\log t$
EncodeC	$2t - 2$	0	0
Cmp $_{<}$ and Cmp $_{=}$	$< 2t$	1	$\log t$
CmpC $_{<}$ and CmpC $_{=}$	$2t - 2$	0	0
DigitDecomp	$4 \frac{L}{\log t}$ (online), $< 6 \frac{tL}{\log t}$ (offline)	$3 \frac{L}{\log t} + 1$	$3L + 2 \frac{L}{\log t}$
DigitDecompC	$6 \frac{tL}{\log t}$	$\frac{L}{\log t} + 1$	$L + \frac{L}{\log t}$

· Without loss of generality, we assume that t is a power of 2. The suffix “C” indicates the use of EncodeC.

Algorithm 10 $\llbracket v'_i \rrbracket_{i \in [n]} \leftarrow \text{IntRadixSort}(t, \llbracket v_i \rrbracket_{i \in [n]})$

Ensure: $\llbracket v'_i \rrbracket_{i \in [n]}$ is sorted array of the inputs.

- 1: **for** $i = 0$ to $n - 1$ **do parallel**
- 2: $\llbracket \tilde{d}_i \rrbracket \leftarrow \text{DigitDecomp}(t, \llbracket v_i \rrbracket)$ ▷ Decompose v_i into k digits in base t
- 3: **end for**
- 4: $\llbracket \mathbf{d}'_i \rrbracket_{i \in [n]} \leftarrow \text{VecRadixSort}(t, \llbracket \mathbf{d}_i \rrbracket_{i \in [n]})$
- 5: **for** $i = 0$ to $n - 1$ **do**
- 6: $\llbracket v'_i \rrbracket \leftarrow \sum_{j=0}^{k-1} t^j \cdot \llbracket d'_{i,j} \rrbracket$
- 7: **end for**
- 8: **return** $\llbracket v'_i \rrbracket_{i \in [n]}$

A. Theoretical Analysis

In this subsection, we present the complexity of the core protocols of UFO_s and of the derived secure integer sorting protocols. Recall that we denote the bit length of each input integer by L , the number of parties by m , and the number of elements to be sorted by n . We use $[t]$ to denote both the domain of one-hot encoding/secure comparison and the base for digit decomposition.

Core protocols of UFO_s. We report the concrete counts of the main secure operations for the two encoding protocols (Encode and EncodeC) as well as the comparison (Cmp $_{<}$, Cmp $_{=}$) and digit-decomposition (DigitDecomp) protocols derived from them. Table I details these counts in terms of the core primitives. The overhead of the main secure operations in the small-element MPC protocols provided in UFO_s scales linearly with the integer bit-length L , yielding high efficiency. Notably, the linearity holds not only asymptotically but also with a very small constant factor.

Compared to Reistad et al.’s bit decomposition [9], which requires $9.5L$ secure multiplications, our total number of multiplications is higher. However, the online phase using Encode requires only $\frac{4}{\log t}L$ secure multiplications, enabling faster execution for larger t . Experiments show that, at $t = 8$, our protocol is approximately three times faster (see section VII-B). Compared with Ning et al. [17], which also supports arbitrary-base decomposition but requires more than $300L$ secure multiplications, our online-phase advantage is even more pronounced.

Sorting. For integer sorting, Table II reports the asymptotic complexity of our IntRadixSort protocol in terms of computation, communication, and rounds. Its complexity depends on the Shuffle protocol, denoted by $\mathcal{S}$.

In the complexity expression, the term $LS/\log t$ decreases with increasing t , whereas $tLmn/\log t$ (or $t^2Lmn/\log t$) grows accordingly. This reflects that a larger t reduces the number of Shuffle invocations but increases the cost of operations related to one-hot encoding, whose overhead scales with t . Such dependence on t introduces additional flexibility, allowing the protocol’s performance to be tuned according to practical requirements.

B. Experimental Evaluation

We evaluate the performance of our proposed protocols on a host equipped with 32 processors, each being Intel(R) Xeon(R) Gold 5122 CPU @ 3.60GHz. The operating system is Ubuntu 18.04.6 LTS. Each party is simulated as a separate process on the host, and the communication is carried out via message passing interface (MPI). To emulate a WAN setting, each pair of parties is connected with a bandwidth of 50 MB/s and an RTT of 20 ms.

We implement our protocols in C++, building upon the Shamir secret sharing scheme. Our experiments are conducted assuming $m = 5$ parties. We implement our own protocols with statistical security parameter $\kappa = 32$ in 128-bit prime field. For previous works that do not require redundancy in field, we implement them in 32-bit prime field, which suffices for their parameter settings. The source code of the experiments is publicly available¹.

2^d-Decomposition. We compare our DigitDecomp protocol with the state-of-the-art bit decomposition method by Reistad et al. [9], which can be extended to a 2^d-decomposition (i.e., representing a number in base 2^d) by combining consecutive bits at low cost. The experimental results are presented in Table III. Our protocol incurs higher offline overhead but significantly outperforms the baseline in the online phase for bases larger than 2. For instance, selecting base 32 reduces the online run time to 0.26ms and communication to 0.272KB, achieving over a fourfold improvement compared to the baseline scheme. As online efficiency is typically critical in practice, these results underscore the practical advantages of our approach.

Digit Decomposition. Table IV demonstrates that, compared to the digit decomposition scheme by Ning et al. [17], our protocol achieves dramatic improvements in the online phase: run time decreases from 23.86–42.99 ms to 0.43–0.74

¹<https://anonymous.4open.science/r/SelectionVectorMPC-A1E8>

TABLE II: Asymptotic complexity of the proposed integer-sorting protocols

Protocol	computation	communication	rounds
IntRadixSort	$O\left(\frac{L(S+tmn)}{\log t}\right)$	$O\left(\frac{L(S+tmn)}{\log t}\right)$	$O\left(\frac{L}{\log t}S + \log \log t + c\right)$
IntRadixSortC	$O\left(\frac{L(S+t^2mn)}{\log t}\right)$	$O\left(\frac{L(S+tmn)}{\log t}\right)$	$O\left(\frac{L}{\log t}S + c\right)$

· The suffix “C” in the protocol name denotes the use of EncodeC.

TABLE III: Performance of 2^d -decomposition on 30-bit integers

Protocol	Base	Run time (ms)		Comm (KB)		Rounds	
		online	offline	online	offline	online	offline
[9]	2	1.14	4.10	1.226	5.052	10	2
Our DigitDecomp	2	1.21	6.90	1.342	8.508	11	4
	8	0.42	5.71	0.420	7.356	11	5
	32	0.26	9.33	0.272	12.532	11	6
	128	0.25	23.84	0.218	32.796	11	6

· “Comm” denotes the communication volume per party.

TABLE IV: Performance of digit decomposition on a 30-bit integers

Base	Protocol	Run time (ms)		Comm (KB)		Rounds	
		online	offline	online	offline	online	offline
5	[17]	42.99	22.58	21.89	18.11	65	16
	Our DigitDecomp	0.74	18.16	0.55	22.48	11	7
20	[17]	23.86	24.42	4.75	19.00	57	16
	Our DigitDecomp	0.51	31.39	0.28	40.92	11	9
80	[17]	42.95	20.38	20.82	17.46	81	16
	Our DigitDecomp	0.43	71.91	0.21	96.22	11	9
320	[17]	25.41	19.34	4.97	18.16	65	16
	Our DigitDecomp	0.46	203.10	0.16	275.21	11	11

· “Comm” denotes the communication volume per party.

ms, corresponding to a 50–100× speedup, while online communication is reduced by 90–97% (0.16–0.55 KB versus 4.63–21.89 KB). The round complexity is also substantially lower, with fixed 11 online rounds and 7–11 offline rounds, compared to 57–81 online and 16 offline rounds in the baseline. Notably, for small t values (e.g., $t = 5$ or 20), our total run time is superior, as the prior method does not optimize for t and operates over the entire domain, resulting in time overheads largely independent of t .

Integer Sorting. We implemented three representative secure sorting methods that were previously analyzed in Section II: Odd-even merge sort [32] (oblivious, $O(n \log^2 n)$), Quick sort [38] (shuffle-then-sort, $O(n \log n)$), and Hamada et al.’s Radix sort [20], which represents the current state-of-the-art MPC secure integer sorting method. We use the shuffle protocol of [40] to implement the shuffle operation required by [20, 38] and our radix sort. Across all tested data scales, our IntRadixSort protocol demonstrates substantial performance gains. The experimental results are shown in Table V. For 65,536 elements, with comparable offline costs, the online run time is 45.20s, compared to 2,623.77s for the baseline radix sort—a 58× improvement. Online communication is reduced to 99.08MB, only 5.4% of the SOTA baseline. The online round count is fixed at 184 and offline rounds at 6, far lower than competing schemes. As data scales increase, these

advantages become more pronounced, with other methods failing to complete for large inputs, highlighting the efficiency and practicality of our protocol.

VIII. CONCLUSION

In this paper, we present UFO_s, an ultra-fast toolkit for multiparty computation on small elements. We design and implement highly efficient protocols for fundamental operations including one-hot encoding, comparison, and digit decomposition. As a showcase application, we develop a novel radix sort protocol leveraging these optimized primitives. Our experiments demonstrate that UFO_s achieves significant performance improvements over the state-of-the-art constructions in various benchmarks. Future directions include extending UFO_s to support a broader range of functionalities and applications, as well as further optimizing the performance and scalability of the protocols.

REFERENCES

- [1] K. Sahinbas and F. O. Catak, “Secure multi-party computation-based privacy-preserving data analysis in healthcare iot systems,” in *Interpretable cognitive Internet of things for healthcare*. Springer, 2023, pp. 57–72.
- [2] P. S. Naidu, R. Kharat, R. Tekade, P. Mendhe, and V. Magade, “E-voting system using visual cryptography

TABLE V: Performance of sorting on 30-bit integers

Num	Protocol	Run time (sec)		Comm (MB)		Rounds	
		online	offline	online	offline	online	offline
4096	Odd-Even Merge sort [32]	1471.35	1097.49	360.29	2121.33	1404	18
	Quick sort [38]	1034.10	146.74	1117.47	582.53	517	15
	Radix sort [20]	183.09	25.14	114.87	86.28	1069	15
	Our IntRadixSort	6.25	28.40	6.19	75.06	184	6
16384	Odd-Even Merge sort [32]	-	-	-	-	-	-
	Quick sort [38]	-	-	-	-	-	-
	Radix sort [20]	671.31	107.39	459.48	345.11	1071	15
	Our IntRadixSort	13.76	109.77	24.77	300.22	184	6
65536	Odd-Even Merge sort [32]	-	-	-	-	-	-
	Quick sort [38]	-	-	-	-	-	-
	Radix sort [20]	2623.77	419.73	1837.91	1380.42	1071	15
	Our IntRadixSort	45.20	442.44	99.08	1200.88	184	6

· “Comm” denotes the communication volume per party. “-” denotes that the experiment did not complete within the time limit.

- & secure multi-party computation,” in *2016 International Conference on Computing Communication Control and automation (ICCUBE)*. IEEE, 2016, pp. 1–4.
- [3] S. S. Cao, L. W. Cong, and B. Yang, “Distributed ledgers and secure multiparty computation for financial reporting and auditing,” *Management Science*, vol. 71, no. 5, pp. 3852–3872, 2025.
- [4] L. Wang, H. Guo, W. Wu, and L. Zhou, “Efficient and privacy-preserving feature selection based on multiparty computation,” *IEEE Transactions on Information Forensics and Security*, vol. 20, pp. 3505–3518, 2025.
- [5] M. Keller, “MP-SPDZ: A versatile framework for multiparty computation,” in *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, 2020.
- [6] P. Mohassel and P. Rindal, “Aby3: A mixed protocol framework for machine learning,” in *Proceedings of the 2018 ACM SIGSAC conference on computer and communications security*, 2018, pp. 35–52.
- [7] I. Damgård and J. B. Nielsen, “Scalable and unconditionally secure multiparty computation,” in *Annual International Cryptology Conference*. Springer, 2007, pp. 572–590.
- [8] O. Catrina and S. De Hoogh, “Improved primitives for secure multiparty integer computation,” in *Security and Cryptography for Networks: 7th International Conference, SCN 2010, Amalfi, Italy, September 13-15, 2010. Proceedings 7*. Springer, 2010, pp. 182–199.
- [9] T. Reistad and T. Toft, “Linear, constant-rounds bit-decomposition,” in *Information, Security and Cryptology-ICISC 2009: 12th International Conference, Seoul, Korea, December 2-4, 2009, Revised Selected Papers 12*. Springer, 2010, pp. 245–257.
- [10] B. Knott, S. Venkataraman, A. Hannun, S. Sengupta, M. Ibrahim, and L. van der Maaten, “Crypten: Secure multi-party computation meets machine learning,” *Advances in Neural Information Processing Systems*, vol. 34, pp. 4961–4973, 2021.
- [11] Y. Wang, G. E. Suh, W. Xiong, B. Lefaudeux, B. Knott, M. Annavaram, and H.-H. S. Lee, “Characterization of mpc-based private inference for transformer-based models,” in *2022 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, 2022, pp. 187–197.
- [12] F. Fu, H. Xue, Y. Cheng, Y. Tao, and B. Cui, “Blindfl: Vertical federated machine learning without peeking into your data,” in *Proceedings of the 2022 International Conference on Management of Data*, 2022, pp. 1316–1330.
- [13] H. Morita, E. Pohle, K. Sadakane, P. Scholl, K. Tozawa, and D. Tschudi, “Maestro: multi-party aes using lookup tables,” in *34th USENIX Security Symposium (USENIX Security 25)*, 2025, pp. 1965–1984.
- [14] T. Nishide and K. Ohta, “Multiparty computation for interval, equality, and comparison without bit-decomposition protocol,” in *Public Key Cryptography-PKC 2007: 10th International Conference on Practice and Theory in Public-Key Cryptography Beijing, China, April 16-20, 2007. Proceedings 10*. Springer, 2007, pp. 343–360.
- [15] T. I. Reistad, “Multiparty comparison-an improved multiparty protocol for comparison of secret-shared values,” in *International Conference on Security and Cryptography*, vol. 1. SCITEPRESS, 2009, pp. 325–330.
- [16] E. Makri, D. Rotaru, F. Vercauteren, and S. Wagh, “Rabbit: Efficient comparison for secure multi-party computation,” in *International Conference on Financial Cryptography and Data Security*. Springer, 2021, pp. 249–270.
- [17] C. Ning and Q. Xu, “Multiparty computation for modulo reduction without bit-decomposition and a generalization to bit-decomposition,” in *International Conference on the Theory and Application of Cryptology and Information Security*. Springer, 2010, pp. 483–500.
- [18] S. Laur, J. Willemson, and B. Zhang, “Round-efficient oblivious database manipulation,” in *Information Security: 14th International Conference, ISC 2011, Xi’an, China, October 26-29, 2011. Proceedings 14*. Springer, 2011, pp. 262–277.
- [19] P. Laud and M. Pettai, “Secure multiparty sorting protocols with covert privacy,” in *Secure IT Systems: 21st Nordic Conference, NordSec 2016, Oulu, Finland,*

- November 2-4, 2016. *Proceedings 21*. Springer, 2016, pp. 216–231.
- [20] K. Hamada, D. Ikarashi, K. Chida, and K. Takahashi, “Oblivious radix sort: An efficient sorting algorithm for practical secure multi-party computation,” *Cryptology ePrint Archive*, 2014.
- [21] K. Chida, K. Hamada, D. Ikarashi, R. Kikuchi, N. Kiribuchi, and B. Pinkas, “An efficient secure three-party sorting protocol with an honest majority,” *Cryptology ePrint Archive*, Paper 2019/695, 2019.
- [22] N. Attrapadung, H. Morita, K. Ohara, J. C. Schuldt, and K. Tozawa, “Memory and round-efficient mpc primitives in the pre-processing model from unit vectorization,” in *Proceedings of the 2022 ACM on Asia Conference on Computer and Communications Security*, 2022, pp. 858–872.
- [23] I. Damgård, M. Fitzi, E. Kiltz, J. B. Nielsen, and T. Toft, “Unconditionally secure constant-rounds multi-party computation for equality, comparison, bits and exponentiation,” in *Theory of Cryptography Conference*. Springer, 2006, pp. 285–304.
- [24] T. Toft, “Constant-rounds, almost-linear bit-decomposition of secret shared values,” in *Cryptographers’ Track at the RSA Conference*. Springer, 2009, pp. 357–371.
- [25] T. Veugen, “Linear round bit-decomposition of secret-shared values,” *IEEE Transactions on Information Forensics and Security*, vol. 10, no. 3, pp. 498–506, 2015.
- [26] H. Chen, H. Li, Y. Wang, M. Hao, G. Xu, and T. Zhang, “Privdt: An efficient two-party cryptographic framework for vertical decision trees,” *IEEE Transactions on Information Forensics and Security*, vol. 18, pp. 1006–1021, 2023.
- [27] P. Xing, H. Li, M. Hao, H. Chen, J. Hu, and D. Liu, “Distributed function secret sharing and applications,” in *NDSS*, 2025.
- [28] D. Rathee, M. Rathee, R. K. Kiran Goli, D. Gupta, R. Sharma, N. Chandran, and A. Rastogi, “Sirnn: A math library for secure rnn inference,” in *2021 IEEE Symposium on Security and Privacy (SP)*, 2021, pp. 1003–1020.
- [29] B. Zhang, “Generic constant-round oblivious sorting algorithm for mpc,” in *International Conference on Provable Security*, X. Boyen and X. Chen, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2011, pp. 240–256.
- [30] A. Agarwal, E. Boyle, N. Chandran, N. Gilboa, D. Gupta, Y. Ishai, M. Kelkar, and Y. Ma, “Secure sorting and selection via function secret sharing,” in *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS ’24. New York, NY, USA: Association for Computing Machinery, 2024, p. 3023–3037.
- [31] K. E. Batchier, “Sorting networks and their applications,” in *Proceedings of the April 30–May 2, 1968, spring joint computer conference*, 1968, pp. 307–314.
- [32] K. V. Jönsson, G. Kreitz, and M. Uddin, “Secure multi-party sorting and applications,” *Cryptology ePrint Archive*, 2011.
- [33] M. T. Goodrich, “Randomized shellsort: A simple data-oblivious sorting algorithm,” *J. ACM*, vol. 58, no. 6, pp. 1–26, Dec. 2011.
- [34] G. Wang, T. Luo, M. T. Goodrich, W. Du, and Z. Zhu, “Bureaucratic protocols for secure two-party sorting, selection, and permuting,” in *Proceedings of the 5th ACM Symposium on Information, Computer and Communications Security*, 2010, pp. 226–237.
- [35] G. Asharov, T. H. Chan, K. Nayak, R. Pass, L. Ren, and E. Shi, “Bucket oblivious sort: An extremely simple oblivious sort,” in *Symposium on Simplicity in Algorithms*. SIAM, 2020, pp. 8–14.
- [36] M. T. Goodrich, “Zig-zag sort: A simple deterministic data-oblivious sorting algorithm running in $O(n \log n)$ time,” in *Proceedings of the forty-sixth annual ACM symposium on Theory of computing*, 2014, pp. 684–693.
- [37] M. Ajtai, J. Komlós, and E. Szemerédi, “An $O(n \log n)$ sorting network,” in *Proceedings of the Fifteenth Annual ACM Symposium on Theory of Computing*, ser. STOC ’83. New York, NY, USA: Association for Computing Machinery, 1983, p. 1–9.
- [38] K. Hamada, R. Kikuchi, D. Ikarashi, K. Chida, and K. Takahashi, “Practically efficient multi-party sorting protocols from comparison sort algorithms,” in *Information Security and Cryptology–ICISC 2012: 15th International Conference, Seoul, Korea, November 28–30, 2012, Revised Selected Papers 15*. Springer, 2013, pp. 202–216.
- [39] D. Bogdanov, S. Laur, and R. Talviste, “A practical analysis of oblivious sorting algorithms for secure multi-party computation,” in *Nordic Conference on Secure IT Systems*. Springer, 2014, pp. 59–74.
- [40] M. Keller and P. Scholl, “Efficient, oblivious data structures for MPC,” in *Advances in Cryptology – ASIACRYPT 2014*, P. Sarkar and T. Iwata, Eds. Springer, 2014, pp. 506–525.
- [41] M. Chase, E. Ghosh, and O. Poburinnaya, “Secret-shared shuffle,” in *Advances in Cryptology–ASIACRYPT 2020: 26th International Conference on the Theory and Application of Cryptology and Information Security, Daejeon, South Korea, December 7–11, 2020, Proceedings, Part III 26*. Springer, 2020, pp. 342–372.
- [42] J. Gao, Y. Zhang, and S. Zhong, “Multiparty shuffle: Linear online phase is almost for free,” *Cryptology ePrint Archive*, Paper 2024/1936, 2024.
- [43] J. Launchbury, I. S. Diatchki, T. DuBuisson, and A. Adams-Moran, “Efficient lookup-table protocol in secure multiparty computation,” *SIGPLAN Not.*, vol. 47, no. 9, p. 189–200, Sep. 2012.
- [44] J. Bar-Ilan and D. Beaver, “Non-cryptographic fault-tolerant computing in constant number of rounds of interaction,” in *Proceedings of the Eighth Annual ACM Symposium on Principles of Distributed Computing*, ser. PODC ’89. New York, NY, USA: Association for Computing Machinery, 1989, p. 201–209.